

Aim:

You are a data scientist at a Bengaluru-based social media startup.

- Users are represented as nodes in a directed graph.
- Edges represent "follows" relationships (e.g., if user A follows user B , there is a directed edge from $A \rightarrow B$).
- When a user posts content, the system must determine the full extent of the post's reach.

However, instead of exploring all immediate followers first (like Breadth-First Search), the algorithm must prioritize following a single chain of interactions to its conclusion before backtracking. This is essentially Depth-First Search (DFS).

Input Format:

- The first line contains an integer n , representing the number of users (nodes).
- The second line contains an integer e , representing the number of follow relationships (edges).
- The next e lines each contain two space-separated integers u and v , representing a directed edge where user u follows user v .
- The last line contains an integer $startUser$, representing the user who makes the post.

Output Format:

- The output prints all users reachable from $startUser$ using DFS traversal.
- Each visited user is printed on a new line in the order they are visited:

<user_id>

- The final line prints the total number of users reached in the format:

Total: <post reach count>

Note: Skeleton code is provided. Complete the **dfs()** function, initialize graph and dfs variables.

Source Code:

postReach.c

```
#include <stdio.h>
#include <stdbool.h>
#define MAX_USERS 100

// graph setup

// dfs variables
int graph[MAX_USERS][MAX_USERS];
bool visited[MAX_USERS];
int totalReach=0;
void dfs(int user, int n) {
    visited[user]=true;
    printf("%d\n",user);
    totalReach++;

    for(int i=0;i<n;i++){
        if(graph[user][i]==1 && !visited[i]){
            dfs(i,n);
        }
    }
}
```

```

    }

    }
    // implement dfs logic

}

int main() {
    int n, e, startUser;
    printf("users: ");
    scanf("%d", &n);
    printf("edges: ");
    scanf("%d", &e);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
        visited[i] = false;
    }
    printf("relationships:\n");
    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }
    printf("user ID: ");
    scanf("%d", &startUser);
    dfs(startUser, n);
    printf("Total: %d\n", totalReach);
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
users:
3
edges:
2
relationships:
0 1
1 2
user ID:
0
0
1
2
Total: 3

Test Case - 2
User Output

users:
5
edges:
3
relationships:
0 1
1 2
3 4
user ID:
0
0
1
2
Total: 3